

Receptive-Field-based Shunting Networks mimic Predictive Coding

Ram J. Zaveri {zaveri@bu.edu}

Boston University

1 Abstract

The objective of this project is to derive synaptic weights and dynamical update rules for receptive-field-based shunting networks. We want to understand whether doing so will yield the famous predictive coding framework originally designed to explain extra-classical receptive fields in the visual cortex. Upon formal derivation, we discuss how modeling the receptive field in a shunting equation as Gaussian Kernels results in a single matrix that, when set to identity, results in the exact formulation as classical predictive coding. We performed supervised learning simulations with three latent layers on the CIFAR-10 dataset. We demonstrated that receptive-field-based shunting networks learn useful representations that accurately recognize objects. Further investigations would include studying receptive fields in the shallow layers and the deeper layers with variable sizes to ensure which formulation is more biologically plausible.

2 Introduction

Shunting networks describe the neuronal dynamics very well. Building on our observations of biological neurons, we can mimic their behavior to some extent to better understand their downstream effects. One such phenomenon is the on-center-off-surround behavior of retinal ganglion neurons in the retina [1]. The exact process in which the neuronal dynamics work is thought to be produced by the famous predictive coding framework [2]. It was designed to explain extra-classical receptive field effects in the visual cortex in the spatial domain [2]. The overall idea is to minimize local errors as a means to minimize total free energy [3]. Interestingly, this results in a local Hebbian-like learning framework that explains how synaptic weights change under various conditions.

In this project, our goal is to utilize our understanding of spatial receptive fields modeled in a shunting equation to derive synaptic update rules along with the update rules for neuronal dynamics. Upon a complete derivation, we hope to converge to the idea that this results in a predictive-coding-like system with lateral connections modeled within the receptive fields as Gaussian kernels. This was indeed shown in our formal derivation, where we also showed that, when the receptive field matrices behave like identity matrices, the resulting formulation is equivalent to the classical predictive coding [4, 2]. Further, we discussed potential drawbacks of our current formulations and how we could improve them to increase biological plausibility.

3 Methods

3.1 Overview

In this section, we discuss how a predictive coding framework could be built from the receptive field-based shunting equations. First, we will discuss the derivation of the shunting equation from physiologically valid assumptions. Then we will incorporate receptive field formulation, as in the on-center-off-surround mechanism. Further, we will discuss the learning rules, propagation behavior, inference, and training details.

3.2 Deriving Shunting Equation

Let us consider a system with the certain properties mentioned below:

1. Limited excitable sites, i.e., B
2. Excitable sites, i.e., x_i , where, i refers to a given system
3. Unexcited sites, i.e., $B - x_i$
4. Expected number of excitable sites turning off, i.e., $(A + \sum_{k \neq i} I_k)x_i$, where $(A + \sum_{k \neq i} I_k)$ is a probability measure influenced by the background noise/inhibition that is coming in
5. Expected number of unexcited sites turning on, i.e., $(B - x_i)I_i$, where I_i is a probability measure

Then, we get the following equation for the rate of change of excitable sites:

$$\begin{aligned} \frac{dx_i}{dt} &= -(A + \sum_{k \neq i} I_k)x_i + (B - x_i)I_i \\ \frac{dx_i}{dt} &= \underbrace{-Ax_i}_{\text{passive decay}} + \underbrace{(B - x_i)I_i}_{\text{shunting excitation}} - \underbrace{x_i \sum_{k \neq i} I_k}_{\text{shunting inhibition}} \end{aligned}$$

We can further derive the equilibrium solution as follows, and simplify $\sum_{k \neq i} I_k \sim L$, $x_i \sim x$, and $I_i \sim I$:

$$\begin{aligned} \frac{dx}{dt} &= -Ax + (B - x)I - xL \\ 0 &= -Ax + BI - xI - xL \\ 0 &= -x(A + I + L) + BI \\ x &= \frac{BI}{A + I + L} \end{aligned}$$

That is the neuron equilibrated for a given input I , decay factor A , and shunting inhibition $\sum_{k \neq i} I_k \sim L$. Further, we can understand this by looking at it from a probabilistic perspective as follows:

$$x = B \cdot \frac{I}{(A + L) + I} \propto \frac{p}{p + q} \quad (1)$$

We can notice how x is proportional to the probability at which the total number of excitable sites is excited, divided by the summation of two probability measures, the probability of the sites being excited, I , and the probability of the sites turning off, $A + L$. The above-mentioned formulation is what we have thoroughly discussed in class. It is important to study neuronal activity at equilibrium, given a constant input influx. We should note that the core assumption is that the input activity is constant for a given neuron. In reality, the inputs do change over time, and with that, the equilibrium also changes. In our simulations, we will assume that the inputs remain consistent for a given neuron for an equilibrium cycle.

3.3 Receptive Field-based Shunting Equation

Now, let's consider this equation as a unit equation, where, if we assume the shunting excitations and inhibitions are both dependent on certain tuning curves of the neurons surrounding them, and are modeled according to Gaussian properties. This will essentially induce a similar effect to the spatial receptive fields. We want to emphasize how neighboring neurons impact the influence of the inputs onto a target neuron, which causes an on-center off-surround effect. This can be achieved by adding two Gaussian kernels that tune for this influence, one to weigh target input contributions and the other to weigh inhibitory contributions. We define them as C and E respectively. The original shunting equation can be modified as below:

$$\frac{dx_i}{dt} = -Ax_i + (B - x_i) \sum_{k=i-n}^{i+n} C_{ki} I_k - x_i \sum_{k=i-n}^{i+n} E_{ki} I_k$$

where receptive field are defined as follows: $C_{ki} = \exp\left(-\frac{\|k-i\|^2}{2\sigma_C^2}\right)$ and $E_{ki} = \frac{1}{\alpha} \cdot \exp\left(-\frac{\|k-i\|^2}{2\sigma_E^2}\right)$. However, we can further simplify this expression by computing "tuning curves" according to the Gaussian kernels for each neuron and storing them in vectors C_i and E_i , respectively. Then we would reorganize and change our equation in the following manner:

$$\frac{dx_i}{dt} = -\gamma x_i + (\beta - x_i) C_i I - x_i E_i I$$

where x_i is a scalar, C_i and E_i are row vectors, and I is a column vector with $\gamma = A$ and $\beta = B$ to ensure the readability of notations in terms of scalars and vectors. $C_i I$ and $E_i I$ will result in a scalar. Now, this entire expression can be vectorized to parallelize the computation in a computer:

$$\frac{d\mathbf{x}}{dt} = -\gamma \mathbf{x} + (\beta - \mathbf{x}) \odot \mathbf{C}I - \mathbf{x} \odot \mathbf{E}I \quad (2)$$

where $\mathbf{C}$ and $\mathbf{E}$ are now matrices, $\mathbf{x}$ and I are column vectors, γ and β are user defined scalars, and $\odot$ represents Hadamard (elementwise) product. Please keep this equation around; we will come back to this later in this paper. To further simplify, we make certain simplifications in Equation 2 and assume our neurons' excitability grows irrespective of the available excitable site for both excitation and inhibition according to some metrics z . Meaning, the excitability and inhibition are both modeled by the same metric with certain Gaussian tuning curves. Therefore, we limit the term $(\beta - \mathbf{x}) \sim z$ in shunting excitation and term $\mathbf{x} \sim z$ in shunting inhibition, and keep the decay dependent only on the current state of the neurons:

$$\begin{aligned} \frac{d\mathbf{x}}{dt} &= -\gamma \mathbf{x} + (\beta - \mathbf{x}) \odot \mathbf{C}I - \mathbf{x} \odot \mathbf{E}I \\ \frac{d\mathbf{x}}{dt} &= -\gamma \mathbf{x} + z \odot \mathbf{C}I - z \odot \mathbf{E}I \\ \frac{d\mathbf{x}}{dt} &= -\gamma \mathbf{x} + z \odot (\mathbf{C}I - \mathbf{E}I) = -\gamma \mathbf{x} + z \odot (\mathbf{C} - \mathbf{E})I \end{aligned}$$

The above manipulations are doable since I is a vector, and matrix-vector multiplications are distributive, but matrix-matrix multiplications aren't always distributive. Additionally, Gaussian kernels have composition properties and can be mathematically modeled in this fashion. Now, $(\mathbf{C} - \mathbf{E})$ is our combined Gaussian Kernel. Let's redefine it as $\mathbf{K} = \mathbf{C} - \mathbf{E}$. Therefore, our equation is:

$$\frac{d\mathbf{x}}{dt} = -\gamma \mathbf{x} + z \odot \mathbf{K}I \quad (3)$$

Equation 3 is our simplified shunting equation with spatial-tuning curves that change with respect to metric z and decay factor γ .

3.4 Deriving Dynamical Rules (Network-level) / Inference Rules

We established our dynamical equation in Equation 3 for a given set of neurons; however, how to evolve that in a network setting is what we discuss in this section. Let's suppose we have $l \in \{0, 1, \dots, L\}$ layers in our network. The deeper layers perform generative inference to predict the activity of the shallow layers and towards the input at $l = 0$. Let's define $I^l = \mathbf{W}^l \mathbf{x}^{l+1}$. The general idea is that if the current layer is able to predict the activity of the shallower layers, then the surprise (error) should be minimal. However, if there is indeed surprise, we only want to update the portion where the surprise occurred, and therefore, we want to compute the surprise, ε , for a given layer l in the following manner:

$$\begin{aligned} \varepsilon^l &= \mathbf{x}^l - \hat{\mathbf{x}}^l \in \mathbb{R}^{d_l} \\ \hat{\mathbf{x}}^l &= \mathbf{K}^l f^l(I^l) \\ &= \mathbf{K}^l f^l(\mathbf{W}^l \mathbf{x}^{l+1}) \end{aligned}$$

where f is an activation function. The total loss for the entire network is computed as the summation of losses at each layer

$$\mathcal{L} = \frac{1}{2} \sum_{l=0}^{L-1} \|\varepsilon^l\|^2$$

The goal is to minimize energy (toward local minima) for a given system, with fixed synaptic weights $\mathbf{W}$. Let's calculate the gradient direction towards the local minima by computing the gradient of the loss, $d\mathcal{L}$, with respect to the neuron activity dx_i . Note that x^l appears in ε^l as well as in ε^{l-1} through $\hat{\mathbf{x}}^{l-1} = f^{l-1}(\mathbf{K}^{l-1}I^{l-1}) = f^{l-1}(\mathbf{K}^{l-1}\mathbf{W}^{l-1}\mathbf{x}^l)$. Therefore, gradients should be computed for ε^l as well as ε^{l-1} for a given loss $\mathcal{L}^l$.

$$\begin{aligned} \frac{d\mathcal{L}}{dx_i^l} &= \frac{d(\frac{1}{2}\|\varepsilon^l\|^2)}{dx_i^l} + \frac{d(\frac{1}{2}\|\varepsilon^{l-1}\|^2)}{dx_i^l} \\ &= \frac{1}{2} \cdot 2 \cdot d\varepsilon^l \frac{\varepsilon^l}{dx_i^l} + \frac{1}{2} \cdot 2 \cdot d\varepsilon^{l-1} \frac{\varepsilon^{l-1}}{dx_i^l} \\ &= \varepsilon^l \frac{d\varepsilon^l}{dx_i^l} + \varepsilon^{l-1} \frac{d\varepsilon^{l-1}}{dx_i^l} \\ &= \varepsilon^l \frac{d(\mathbf{x}^l - \hat{\mathbf{x}}^l)}{dx_i^l} + \varepsilon^{l-1} \frac{d(\mathbf{x}^{l-1} - \hat{\mathbf{x}}^{l-1})}{dx_i^l} \\ &= \varepsilon^l - \varepsilon^{l-1} \frac{d\hat{\mathbf{x}}^{l-1}}{dx_i^l} \\ &= \varepsilon^l - \varepsilon^{l-1} \frac{d(\mathbf{K}^{l-1} f^{l-1}(\mathbf{W}^{l-1}\mathbf{x}^l))}{dx_i^l} \\ &= \varepsilon^l - \varepsilon^{l-1} \mathbf{K}^{l-1} f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l)(\mathbf{W}^{l-1}) \end{aligned}$$

Once we rearrange the above derived update, we get the following:

$$\frac{d\mathcal{L}}{dx_i^l} = \varepsilon^l - (\varepsilon^{l-1} \odot \mathbf{K}^{l-1} f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l) \mathbf{W}^{l-1}) \quad (4)$$

Following that, our update rule for inference with respect to the computed gradient is as follows:

$$\frac{d\mathbf{x}^l}{dt} = -\gamma \mathbf{x}^l - \frac{d\mathcal{L}}{dx_i^l} = -\gamma \mathbf{x}^l + (\varepsilon^{l-1} \odot \mathbf{K}^{l-1} f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l) \mathbf{W}^{l-1}) - \varepsilon^l \quad (5)$$

From our Equation 3, $\frac{d\mathbf{x}}{dt} = -\gamma \mathbf{x} + z \odot \mathbf{K}I$, we can realize why substituting $\mathbf{x}$ with z would provide us with a better simplification of the network. The decay rate for a given neuron depends on the neuron's state itself, $-\gamma \mathbf{x}$. $\mathbf{K}$ modulates weights according to the lateral inhibition/competitiveness in a receptive-field-based manner. I could be categorized as the propagation of input to change the activity of the neuron. Therefore, $\mathbf{K}I$ can be replaced by the nonlinear term from Equation 5, $\mathbf{K}^{l-1} f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l) \mathbf{W}^{l-1}$, where $f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l) \mathbf{W}^{l-1}$ is essentially the derivative of $f(I)$ with respect to $\mathbf{x}$. z could be replaced with ε^{l-1} , which is the errors from the layer below, influencing the impact of the neuron at the current layer. ε^l is the error at the current layer we want to minimize. This step is called the inference step and can be summarized in the following equation:

$$\frac{d\mathbf{x}^l}{dt} = -\gamma \mathbf{x}^l - \eta_{inference} \left(\varepsilon^l - (\varepsilon^{l-1} \odot \mathbf{K}^{l-1} f^{(l-1)'}(\mathbf{W}^{l-1}\mathbf{x}^l) \mathbf{W}^{l-1}) \right) \quad (6)$$

where $\eta_{inference}$ is the inference rate. Iteratively, as in via Euler integration, we achieve an equilibrium state for this dynamical system with fixed synaptic weights $\mathbf{W}$. Once the equilibrium state is reached, we can analyze the network's activity and update the synaptic weights accordingly. This procedure is known as the alternate optimization [4] or expectation-maximization algorithm [5]. In practice, we set a specific number of steps this dynamical system can take. Additionally, deriving an analytical solution is not possible because the dynamical system has a nonlinear component, which makes it difficult to understand the dynamical system's curve unless it is numerically resolved over time.

3.5 Synaptic Weight Updates

Let's derive the update rules for the synaptic weights of our network. We want to minimize $\mathcal{L}$ with respect to the synaptic weights $\mathbf{W}$. Therefore, the derivation is as follows:

$$\begin{aligned}
\frac{d\mathcal{L}}{d\mathbf{W}^l} &= \frac{d(\frac{1}{2}\|\varepsilon^l\|^2)}{d\mathbf{W}^l} \\
&= \frac{1}{2} \cdot 2 \cdot d\varepsilon^l \frac{\varepsilon^l}{d\mathbf{W}^l} \\
&= \varepsilon^l \frac{d\varepsilon^l}{d\mathbf{W}^l} \\
&= \varepsilon^l \frac{d(\mathbf{x}^l - \hat{\mathbf{x}}^l)}{d\mathbf{W}^l} \\
&= -\varepsilon^l \frac{d\hat{\mathbf{x}}^l}{d\mathbf{W}^l} \\
&= -\varepsilon^l \frac{d(\mathbf{K}^l f^l(\mathbf{W}^l \mathbf{x}^{l+1}))}{d\mathbf{W}^l} \\
&\sim -\varepsilon^l \mathbf{K}^l f^{l'}(\mathbf{W}^l \mathbf{x}^{l+1})(\mathbf{x}^{l+1}) \\
\frac{d\mathcal{L}}{d\mathbf{W}^l} &= -(\mathbf{x}^{l+1})^T \left[\mathbf{K}^l f^{l'}(\mathbf{W}^l \mathbf{x}^{l+1}) \odot \varepsilon^l \right]
\end{aligned}$$

where, $f^{l'}(\mathbf{W}^l \mathbf{x}^{l+1})$ is generally referred to as gain [4]. However, if we ignore the term $\mathbf{K}^l f^{l'}(\mathbf{W}^l \mathbf{x}^{l+1})$ for now, we would get

$$\begin{aligned}
\frac{d\mathcal{L}}{d\mathbf{W}^l} &= -(\mathbf{x}^{l+1})^T \left[\varepsilon^l \right] = -(\mathbf{x}^{l+1})^T \underbrace{\left[\mathbf{x}^l - \hat{\mathbf{x}}^l \right]}_{\substack{\text{surprise/error} \\ \text{Hebbian Growth}}}
\end{aligned}$$

Now, our weight update is as follows:

$$\frac{d\mathbf{W}^l}{dt} = -\gamma_w \mathbf{W} - \eta_{train} \frac{d\mathcal{L}}{d\mathbf{W}^l} = -\gamma_w \mathbf{W} + \eta_{train} \left((\mathbf{x}^{l+1})^T \left[\mathbf{x}^l - \hat{\mathbf{x}}^l \right] \right)$$

where the second term is the Hebbian growth term we studied in class and η_{train} is learning rate. In practice we use the following equation to perform the update:

$$\frac{d\mathbf{W}^l}{dt} = -\gamma_w \mathbf{W} - \eta_{train} \frac{d\mathcal{L}}{d\mathbf{W}^l} = -\gamma_w \mathbf{W} + \eta_{train} \left((\mathbf{x}^{l+1})^T \left[\mathbf{K}^l f^{l'}(\mathbf{W}^l \mathbf{x}^{l+1}) \odot \varepsilon^l \right] \right) \quad (7)$$

These expressions show how local synaptic learning can be achieved, which depends only on the pre-synaptic and post-synaptic neurons. We should also note that $\mathbf{K}$ can also be parameterized for better performance or removed altogether. In practice, larger networks focus only on presynaptic-to-postsynaptic connections and ignore lateral (given by $\mathbf{K}$ in our framework) connections, as this has been shown to improve simulation performance.

4 Results

4.1 Implementation Details

This work takes advantage of formulations from [4], and keeps the architecture and hyperparameters consistent with those. Our model has three latent layers where $\mathbf{x}^l \in \{1000, 500, 10\}, l \in \{1, 2, 3\}$, and one observation layer for CIFAR10 dataset [6], i.e., $\mathbf{x}^{l=0} = 3072 = 3 \times 32 \times 32$. Between every layer, we have a weights matrix $\mathbf{W} \in \{\mathbb{R}^{3072 \times 1000}, \mathbb{R}^{1000 \times 500}, \mathbb{R}^{500 \times 10}, \mathbb{R}^{10 \times 10}\}$. Therefore, this network has about $3,072 \times 1,000 + 1,000 \times 500 + 500 \times 10 + 10 \times 10 = 3,577,100 = 3.6$ million connections/parameters. The Gaussian kernel variances are as follows: $\sigma_C^2 = 0.05$ and $\sigma_E^2 = 0.1$ with $\alpha = 10$. We also perform batch training where the batch size is kept at 500 samples per batch, where we have 100 batches for training

the algorithm and 20 in the test set. For Euler integration in achieving the equilibrium for each neuronal population (the inference step), we perform 50 iterations and use the values at the 50th iteration as the equilibrated value. We should note that equilibrium might not have been achieved at the 50th iteration; however, for efficiency and given the number of parameters, we keep it that way. To run the simulations, we use Python 3.10 with PyTorch [7]. We also run the simulations only for one epoch, where the network sees the data only once.

4.2 Simulation Dynamics

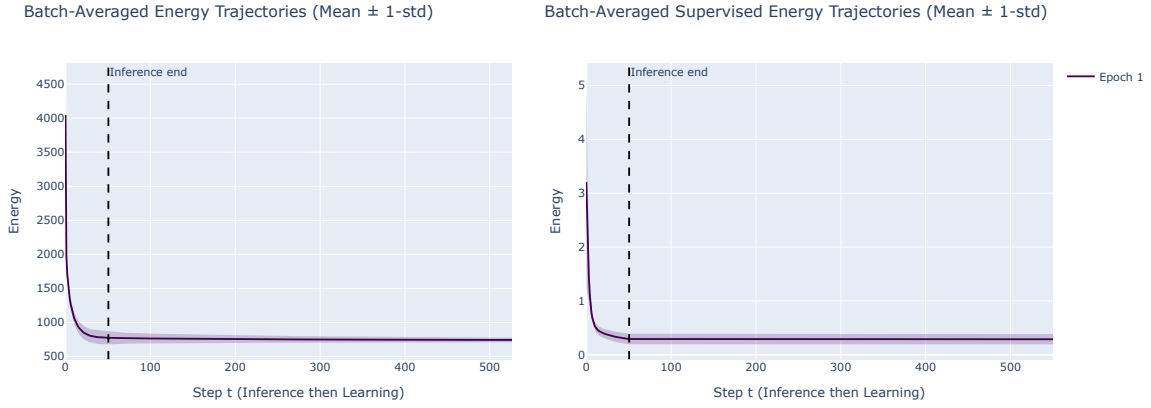


Figure 1: **Energy/Surprise/Error trajectories over time for one epoch.** Left: average energy values for latent for layers $l \in \{0, 1, \dots, L - 1\}$. Right: average energy values for the supervised loss at layer $l = L$.

In Figure 1, we show error (surprise/energy) values for the entire training set. The first portion indicates the Euler integration for neurons, while the rest indicates the weight updates. It should be noted that the values are averaged across batches. The left figure describes the average energy values for latent layers $l \in \{0, 1, \dots, L - 1\}$ and the right describes average energy values for the supervised loss at layer $l = L$. As we can see, in the first ~ 50 time-steps, the errors quickly decrease, indicating the neurons' equilibrating behavior. Supervise loss indicates its ability to recognize the object. This simulation shows that the network is exhibiting the expected behavior. Next, we will validate this learning by calculating the accuracy obtained on the test set.

4.3 Simulation Results

Table 1: Top-1 and Top-3 accuracies on CIFAR-10 [6] dataset. The left portion suggests the mean, and the right suggests the standard deviations across five runs. Bold suggests higher values.

Accuracy	Receptive-Field-based Predictive Coding	Regular Predictive Coding
Top-1 %	98.614 ± 0.933 (%)	95.638 ± 6.456 (%)
Top-3 %	99.816 ± 0.093 (%)	99.038 ± 2.034 (%)

In Table 1, we illustrate the simulation results obtained across five runs. The left portion suggests the mean, and the right suggests the standard deviations. We show both Top-1% and Top-3% accuracies on CIFAR-10 [6] dataset. Here, we compare it with a conventional predictive coding framework [4], which disregards lateral connections. The only modification in the derived equation would be setting $\mathbf{K}^l = \mathbf{I}^l$ to be an identity matrix. As shown in Table 1, the Receptive-Field-based framework achieves better performance than the conventional version. The performance loss in the conventional method is due to the inherent instability of the predictive coding framework. Utilizing Gaussian kernels as a means of lateral connections could potentially aid in stabilizing the network; however, more studies are needed on this point.

5 Discussion

In this project, we showcased a full derivation of Receptive-Field-based shunting networks for inference and learning rules with certain assumptions and simplifications. We performed the task of object recognition on CIFAR-10 [6] and showed that Receptive-Field-based networks could, in fact, learn useful generative features to recognize objects. However, we should note that the receptive field variances were kept very small ($\sigma_C^2 = 0.05$ and $\sigma_E^2 = 0.1$ with $\alpha = 10$), and lateral connections were minimal in this network. This set of parameters worked the best given the time constraints we had. While increasing the receptive-field size with various variance values, we noticed very unstable behavior of our system, and it did not minimize the errors; instead, those values exploded to infinity. We hypothesize that this may be due to the vector summation, which causes the incoming input weights to fluctuate too high, resulting in errors exploding. We believe that adding a normalization term by parameterizing the kernel variances could help minimize this behavior. Additionally, receptive-field-based systems might be prominent only in shallower layers, not in deeper ones. It might help perform an ablative analysis on receptive fields for shallower networks versus the deeper ones. Given the time constraints, we were unable to perform this analysis. However, we will continue working on this in the future to better understand the behavior and develop more complex networks.

This investigation helped us develop the necessary skills to understand dynamical equations, derive formulations such as update rules to achieve equilibrium for neurons and synaptic weights. Utilizing various components from computational neuroscience to come up with a theory that either supports or refutes the biological phenomena is one goal of this exercise. We did that by understanding spatial tuning curves in biological systems and first implementing those in shunting equations. We further showed how inference rules associate with the shunting equation, and weight update rules follow biological plausibility. This project helped me develop a better understanding of neuronal dynamics, propagation, update rules, and predictive coding.

6 Conclusion

To conclude, this project helped me develop a deeper understanding of the predictive coding framework for modeling Gaussian-kernel-inspired receptive-field-based shunting networks. These networks not only have layer-by-layer connections but also lateral connections that induce the on-center-off-surround effect. Having to derive the inference and learning rules helped me develop a skill set that will be useful for modeling and simulating other biological behaviors in a complex in-silico system. As my interests lie in understanding and producing representational alignment between in-silico and biological neural networks, this project has been a great step-up in that direction.

7 Acknowledgment

We thank Dr. Arash Yazdanbakhsh for his constant engagement in class and for consistently providing positive, helpful feedback. We also want to acknowledge that this work is heavily influenced by [4] where they also made their codebase available for anyone to use.

References

- [1] T. Hosoya, S. A. Baccus, and M. Meister, “Dynamic predictive coding by the retina,” *Nature*, vol. 436, no. 7047, pp. 71–77, 2005.
- [2] R. P. Rao and D. H. Ballard, “Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects,” *Nature neuroscience*, vol. 2, no. 1, pp. 79–87, 1999.
- [3] K. Friston, “A theory of cortical responses,” *Philosophical transactions of the Royal Society B: Biological sciences*, vol. 360, no. 1456, pp. 815–836, 2005.
- [4] M. Stenlund, “Introduction to predictive coding networks for machine learning,” *arXiv preprint arXiv:2506.06332*, 2025.

- [5] R. A. Boyles, "On the convergence of the em algorithm," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 45, no. 1, pp. 47–50, 1983.
- [6] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.